TiendaYa

Informe de arquitectura final — Entrega final

Bases de Datos 2

Barrios Cruz, Mynor Enrique
Universidad del Istmo — UNIS
Guatemala, 30 de octubre de 2026

Índice

1. Introducción y alcance de la entrega final 3
2. Arquitectura final 4
3. Recomendación y búsqueda semántica 5
3.1 Problema 5
3.2 Decisión (ADR-004) 5
3.3 Cómo funciona 5
3.4 Funciones 6
3.5 Evaluación de calidad 6
3.6 Escala: 100 000 y 1 000 000 de vectores 7
3.7 Cuándo migrar a una base vectorial dedicada 7
3.8 Si algo falla 8
4. Control de acceso 8
4.1 Problema 8
4.2 Decisión (ADR-005) 8
4.3 Usuarios y permisos 8
4.4 Validación 9
4.5 Evidencia 9
4.6 Limitaciones asumidas 9
5. Retrospectiva 10
6. Verificación y presentación 10
7. Conclusiones 11
Anexo. Documentos y comandos 11

1. Introducción y alcance de la entrega final

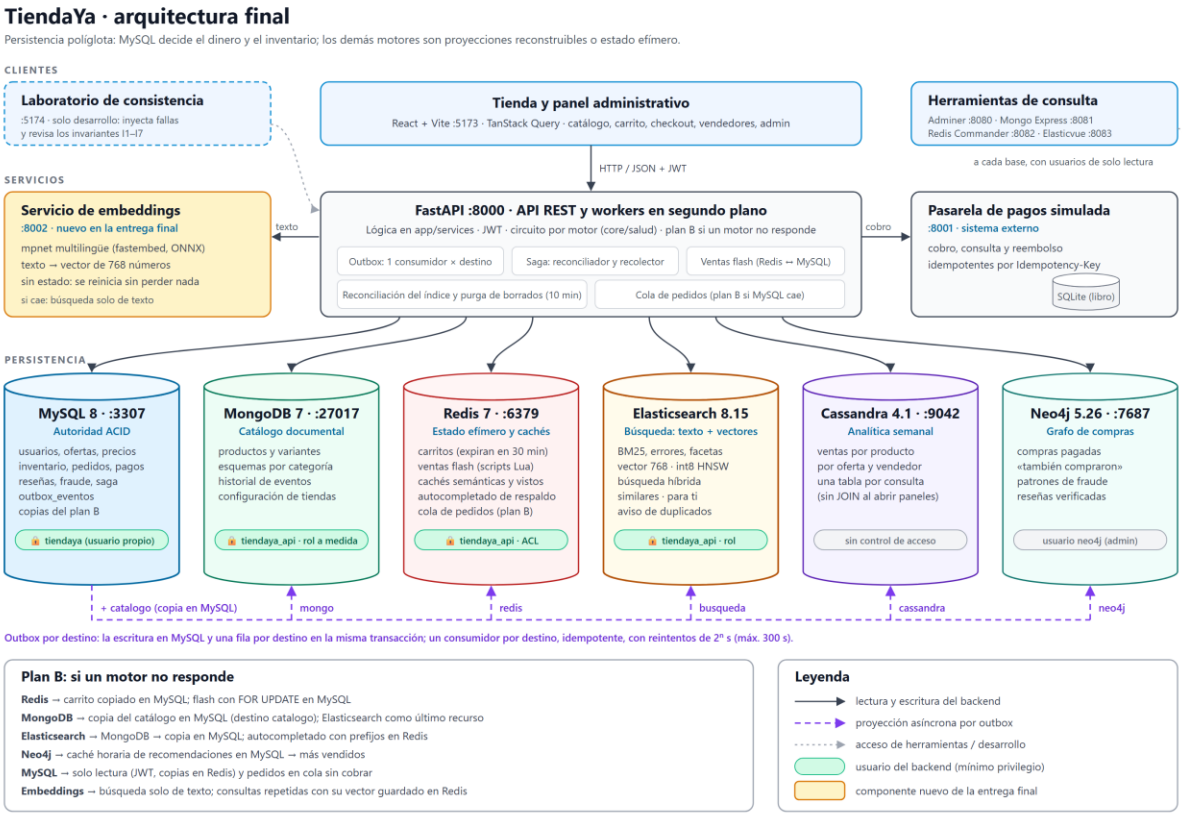
TiendaYa es un marketplace con persistencia políglota: cada motor de base de datos resuelve el problema para el que está mejor equipado. Las entregas anteriores incorporaron MySQL como autoridad transaccional, MongoDB para el catálogo documental, Redis para carritos y ventas flash, Cassandra y Neo4j para analítica y fraude, y Elasticsearch para la búsqueda, con un checkout distribuido como saga.

La entrega final cierra la arquitectura con dos capacidades que suelen postergarse y que el enunciado pide de forma explícita, más los documentos de cierre:

Criterio del enunciado	Peso	Qué se entrega	Sección
Recomendación semántica	5 pts	Vectores de cada producto en Elasticsearch y un servicio de embeddings propio: «También te podría interesar» en la página de producto, «Te podría interesar» en el inicio, búsqueda híbrida y aviso de duplicados	3
Control de acceso en un componente NoSQL	4 pts	Usuarios por servicio con mínimo privilegio en Redis (ACL), MongoDB (roles) y Elasticsearch (roles), con evidencia automática	4
Diagrama de arquitectura final	3 pts	Todos los componentes de persistencia y sus relaciones	2
Retrospectiva	4 pts	Qué decisiones se sostendrían y cuáles se replantearían, con justificación	5
Presentación	4 pts	Modo presentación con los datos DEMO, ya con vectores y control de acceso	6

Cada decisión se tomó con evidencia medida: una línea base antes de cambiar, una evaluación con consultas etiquetadas a mano, un benchmark de escala y pruebas automáticas que se pueden volver a ejecutar. Las decisiones se registran como ADR con la estructura del curso (contexto, alternativas, decisión, justificación y consecuencias).

2. Arquitectura final



La regla que organiza la arquitectura: MySQL decide el dinero y el inventario; los demás motores son proyecciones reconstruibles o estado efímero. Toda escritura autoritativa ocurre en MySQL junto con una fila de outbox por destino, en la misma transacción; un consumidor por destino proyecta hacia MongoDB, Elasticsearch, Cassandra, Neo4j, Redis y la copia del catálogo, de forma idempotente y sin que un destino caído retrase a los demás.

Componente	Responsabilidad	Usuario del backend
MySQL 8	Usuarios, ofertas, precios, inventario, pedidos, pagos, reseñas, fraude, saga y outbox. Única fuente de verdad; guarda también las copias del plan B	tiendaya (usuario propio)
MongoDB 7	Productos, variantes, esquemas de atributos por categoría, historial de eventos	tiendaya_api · rol a medida
Redis 7	Carritos con expiración, ventas flash (Lua), autocompletado y cola del plan B, cachés de la búsqueda semántica e historial de vistos	tiendaya_api · ACL
Elasticsearch 8.15	Búsqueda de texto y vector de cada producto (dense_vector int8 HNSW); decide qué productos y en qué orden; precio y stock se hidratan desde MySQL	tiendaya_api · rol
Cassandra 4.1	Ventas pagadas por producto, oferta, vendedor y semana	(sin control de acceso)
Neo4j 5.26	Compras pagadas: «también compraron» y patrones	neo4j

	de fraude	
Servicio de embeddings	Convierte texto en vectores de 768 números (modelo local, sin estado)	—
Pasarela simulada + SQLite	Cobro, consulta y reembolso idempotentes por clave	—

El documento docs/05-diagrama-arquitectura.md incluye además tres vistas editables: componentes y usuarios, flujo de autoridad → proyecciones → respaldos, y el recorrido de los vectores.

3. Recomendación y búsqueda semántica

3.1 Problema

Se midió el sistema antes de cambiarlo, con 35 consultas etiquetadas a mano (exactas, con errores o sinónimos y en lenguaje natural):

- La búsqueda de texto no entiende lenguaje natural: «herramienta para hacer agujeros en la pared» no encuentra el taladro. En esas consultas, P@5 = 0.07 y MRR = 0.23.
- «Quienes compraron también compraron» (Neo4j) no recomienda nada para productos sin ventas.
- La similitud por palabras (more_like_this) no encuentra nada para el 43 % de los productos.
- Las solicitudes de catálogo no detectan duplicados escritos distinto («WH1000XM5» frente a «WH-1000XM5»).

3.2 Decisión (ADR-004)

Alternativa	Por qué no / por qué sí
No introducir cambios	No cumple el requerimiento y conserva los problemas medidos
Extensión relacional (pgvector, MySQL 9 VECTOR)	MySQL 8 no tiene tipo vectorial; pgvector exige sumar PostgreSQL solo para vectores; MySQL 9 comunitario no tiene índice aproximado
Base vectorial dedicada (Qdrant, Milvus)	Un motor más, un destino más del outbox y una segunda consulta para combinar con el texto; sin beneficio medible a esta escala (ver 3.7)
Redis Stack / MongoDB Atlas Vector Search	Exigen cambiar la distribución de motores que ya cumplen otro rol
Elasticsearch dense_vector (elegida)	Ya está en la arquitectura: kNN con filtros y combinado con el texto en una sola consulta, con los mismos filtros y facetas

Los vectores los calcula un servicio de embeddings propio (FastAPI, puerto 8002) con el modelo multilingüe paraphrase-multilingual-mpnet-base-v2 (768 dimensiones), que se ejecuta localmente con ONNX. Se comparó con MiniLM-L12 (384 dimensiones): MRR 0.74 frente a 0.66.

3.3 Cómo funciona

- El vector viaja dentro del documento de búsqueda (mapping v2): dense_vector de 768 dimensiones, similitud coseno, índice HNSW cuantizado a int8 (cuatro veces menos memoria).
- Solo se recalcula si cambia el texto del producto: una huella sha256 de «modelo + texto» permite reutilizar el vector en los reindexados por precio o stock.

- Viaja por el mismo outbox que el resto del documento: no puede quedar un vector de un producto inactivo ni de otra versión.
- Si el servicio no responde, el producto se indexa sin vector y la reconciliación periódica lo completa.
- Hallazgo corregido: tras muchas reindexaciones, las versiones borradas de los documentos estorbaban al grafo HNSW y el kNN llegaba a devolver cero resultados. Se purgan cuando pasan del 20 %.

3.4 Funciones

Función	Dónde	Cómo y por qué
«También te podría interesar:»	Página de producto, debajo de «Las personas que han comprado este artículo también han comprado estos otros:»	kNN del vector del producto; excluye lo que ya muestra la sección de compras. Las compras sugieren complementos; los vectores, alternativas, y funcionan sin ventas. Caché de una hora en Redis
Búsqueda híbrida	Catálogo	Texto + kNN en una consulta. Las coincidencias de texto estrictas van primero; el vector las reordena y agrega relacionados (similitud ≥ 0.45). Sin coincidencias, primero lo relacionado por significado y después los populares
«Te podría interesar»	Inicio	Una cláusula kNN por semilla (carrito pesa 2; vistos 1, 0.8, 0.64…) en vez de promediar vectores: el promedio de una laptop y unos audífonos no se parece a ninguno
Aviso de duplicados	Solicitudes de catálogo (vendedor y administrador)	kNN más parecido del nombre; dos niveles (posible / muy probable); nunca bloquea

3.5 Evaluación de calidad

69 productos, 35 consultas etiquetadas. P@5: relevantes entre los 5 primeros; MRR: 1 / posición del primer relevante.

Método	P@5 (todas)	MRR (todas)	MRR exactas	MRR lenguaje natural
Solo texto (Entrega 3)	0.24	0.62	1.00	0.23
Solo vectores	0.33	0.74	0.93	0.69
Vectores con umbral 0.45	0.25	0.68	0.93	0.66
Híbrida (en producción)	0.34	0.82	1.00	0.67

Productos similares	Mismo departamento (P@5)	Misma familia (P@5)	Productos con resultados
more_like_this (palabras)	0.29	0.14	57 %
kNN (vectores)	0.72	0.28	100 %

- La evaluación cambió una decisión: con la base fija para toda coincidencia de texto, «novela de fantasía» ponía primero el «Nivel» láser (coincidencia con error). Limitar la base a coincidencias estrictas subió el MRR de las consultas naturales de 0.61 a 0.67 sin perder nada en las exactas.

- El umbral 0.45 es el más bajo con el que ninguna de 6 consultas sin sentido devuelve resultados.
- HNSW + int8 coincide en un 97.4 % con la búsqueda exacta sobre el catálogo real.

3.6 Escala: 100 000 y 1 000 000 de vectores

Vectores sintéticos de 768 dimensiones agrupados alrededor de los productos reales, en un nodo con 512 MB de heap (una laptop).

Medida	100k int8	100k float32	1M int8
Carga (vectores/s)	1709	1765	1622
Latencia kNN p50 / p95 (cliente)	8.6 / 10.3 ms	8.7 / 11.3 ms	8.8 / 10.8 ms
Con filtro de categoría (1 %) p50	6.4 ms	7.0 ms	14.7 ms
Consultas por segundo (8 hilos)	556.0	551.1	328.3
Memoria del kNN (estimada)	86 MB	305 MB	858 MB
Recall@10 (200 candidatos)	0.85	1.00	0.81
Recall@10 con reordenamiento float32	0.98	—	0.99
Búsqueda exacta sin HNSW	27 ms	30 ms	146 ms

- La latencia casi no cambia de 100 000 a 1 000 000 (HNSW crece de forma logarítmica); sin HNSW, la búsqueda exacta crece de forma lineal (1.5 s proyectados con 10 M).
- int8 pierde exactitud en familias de productos muy parecidas; pedir 5 veces más vecinos y reordenarlos con los vectores float32 recupera un recall de 0.99 por unos 13 ms.
- Límites encontrados: con 512 MB de heap, fusionar 1 M de vectores en un segmento hizo caer el nodo por falta de memoria, y el 70 % del disco es el vector guardado como texto en _source.

3.7 Cuándo migrar a una base vectorial dedicada

Una base vectorial dedicada no es más rápida por sí misma: Elasticsearch ya responde en ~9 ms con un millón de vectores. Su beneficio aparece cuando la carga vectorial compite con la búsqueda de texto o cuando la memoria encarece el clúster. Condiciones, con los números medidos:

Condición	Umbral	Por qué
Memoria de los vectores	~900 bytes por vector con int8: 10 M ≈ 8.6 GB; 50 M ≈ 43 GB por copia	Si no caben en la caché de páginas, el kNN lee de disco
Tráfico semántico sostenido	> ~300 consultas kNN por segundo por nodo (medido: 556 con 100k, 328 con 1M)	Le quita CPU y caché a la búsqueda de texto
Ritmo de escritura	> ~1 600 vectores/s por nodo, o reindexar decenas de millones al cambiar de modelo	Bloquearía la indexación de cambios normales
Mantenimiento	Fusionar 1 M de vectores no cupo en 512 MB de heap	Reconstruir el grafo exige memoria
Funciones	Varios vectores por producto, colecciones por vendedor	Las bases dedicadas están hechas para eso

El primer paso sería separar los vectores en nodos propios de Elasticsearch (la misma tecnología); la arquitectura ya lo permite, porque el outbox tiene un destino por componente. Con 69 productos, y aun con un millón, no se cumple ninguna condición.

3.8 Si algo falla

Falla	Qué pasa
Servicio de embeddings	Las consultas ya hechas usan su vector guardado en Redis; las nuevas, solo texto. Similares e inicio siguen (usan vectores ya indexados). Duplicados: sin aviso
Elasticsearch	Similares → más vendidos de la categoría; búsqueda → MongoDB y plan B; inicio → sin sección
Redis	Sin cachés: se consulta directo

4. Control de acceso

4.1 Problema

Antes de esta entrega Redis no pedía autenticación, el backend se conectaba a MongoDB como root, Elasticsearch tenía la seguridad desactivada y las contraseñas estaban fijas en el repositorio. Un fallo en el backend o cualquier proceso de la máquina tenía control total sobre los tres motores.

4.2 Decisión (ADR-005)

Alternativa	Resultado
No cambiar	No cumple
Solo aislar la red	Cualquier proceso local sigue con control total; el backend sigue siendo root
Una contraseña por motor	Autenticación sin autorización: un fallo del backend puede seguir borrando todo
Usuarios por servicio con mínimo privilegio (elegida)	Autenticación y autorización acotada por servicio
Gestor de secretos externo	Un servicio más; cambia dónde viven las contraseñas, no los permisos

4.3 Usuarios y permisos

Motor	Backend	Herramientas de consulta	Instalación / operación
Redis (ACL)	tiendaya_api: 9 prefijos de llaves y 34 comandos; sin KEYS, FLUSHALL, CONFIG, ACL ni INFO	tiendaya_lectura y default: solo lectura (Redis Commander)	tiendaya_admin
MongoDB (roles)	tiendaya_api: rol a medida sobre 5 colecciones (find, insert, update, remove, createIndex,	tiendaya_lectura: read (Mongo Express)	tiendaya_admin: dbOwner solo de tiendaya

	listIndexes); sin drop ni otras bases		
Elasticsearch (roles)	tiendaya_api: documentos de productos*; no crea ni borra índices ni mueve el alias	tiendaya_lectura: solo leer (Elasticvue)	tiendaya_indexador: productos* y bench_vectores_*

- Reglas versionadas en el repositorio (database/redis/acl.conf, database/mongo/03_usuarios.js, database/elasticsearch/seguridad.json); contraseñas generadas al azar en .env, fuera de git.
- Sin modo abierto: si falta una contraseña, docker compose se niega a arrancar y dice qué ejecutar.
- Las instalaciones existentes adoptan las contraseñas con que se crearon sus volúmenes; un solo comando (configurar-acceso.ps1) configura un equipo, y otro el entorno DEMO de la presentación.

4.4 Validación

Los permisos salen de un inventario de lo que el código hace y se validaron corriendo toda la suite con los usuarios restringidos y revisando los rechazos de cada motor (ACL LOG de Redis, registro de control de acceso de MongoDB, advertencias del backend con Elasticsearch). El plan B podía esconder un permiso faltante —si un motor rechaza, el backend usa el respaldo sin que falle ninguna prueba—; así apareció ZRANGEBYLEX en Redis, que el inventario no tenía.

4.5 Evidencia

Intento	Resultado
Conectarse sin credenciales (Redis, MongoDB, Elasticsearch)	NOAUTH · requires authentication · 401
Backend escribe una llave fuera de sus prefijos en Redis	NOPERM No permissions to access a key
Backend ejecuta FLUSHALL, KEYS, CONFIG o ACL en Redis	NOPERM ... has no permissions to run the command
Script Lua del backend escribe fuera de sus prefijos	ACL failure in script
Backend lee migraciones, admin.system.users o crea un usuario root en MongoDB	not authorized
Backend crea un índice, mueve el alias o crea un superusuario en Elasticsearch	403 security_exception
Usuarios de lectura escriben o borran	NOPERM · not authorized · 403

backend/scripts/verificar_permisos.py hace 40 intentos reales contra el sistema en marcha (40 de 40 dieron lo esperado) y deja la tabla en docs/evidencia/permisos.md; además hay 50 pruebas automáticas. Lo destructivo no se ejecuta: se pregunta al motor si el usuario tendría el privilegio (connectionStatus, has_privileges).

4.6 Limitaciones asumidas

- Sin TLS en desarrollo (motores en la misma máquina); en producción se activaría sin cambiar usuarios ni roles.
- Los superusuarios siguen existiendo, solo para crear los usuarios; nunca los usa el backend.

- Neo4j y Cassandra no se restringieron en esta entrega (MySQL ya usaba un usuario propio).

5. Retrospectiva

Resumen de docs/49-retrospectiva.md: catorce decisiones de las cuatro entregas, evaluadas con los criterios de los ADR y evidencia medida en el proyecto.

Decisión	Veredicto	Justificación
MySQL única autoridad; el resto, proyecciones	Se sostiene	Toda divergencia se corrigió reproyectando desde MySQL; invariantes I1–I7 en cada fase
MongoDB para el contenido del catálogo	Se sostiene	Atributos distintos por categoría, variantes e historial sin tablas EAV
Redis para carritos y ventas flash	Se sostiene	TTL nativo y Lua atómico; el ACL aplica dentro de Lua
Elasticsearch para búsqueda y vectores	Se sostiene	6/17 → 17/17 aciertos; híbrida MRR 0.62 → 0.82; ~9 ms con 1 M
Checkout como saga	Se sostiene	Escenarios de falla E00–E17 sin dinero ni inventario perdidos
Outbox por destino (segunda versión)	Se sostiene	La primera aplicaba deltas: Mongo quedó con stock 9 frente a 16. Una proyección se deriva del estado
Precio y stock copiados en MongoDB	Se replantea	Cuatro copias del mismo dato; ahí ocurrió la falla más visible
Escribir el catálogo en MongoDB y después en MySQL	Se replantea	Escritura doble con ventana de 10 minutos
Neo4j sin reconciliación propia	Se replantea	La divergencia 192/175 persistió toda la entrega
Cassandra desde la Entrega 2	Se replantea el momento	A este volumen bastaban tablas resumen en MySQL
Seguridad al final del proyecto	Se replantea	Tocar decenas de archivos y adoptar contraseñas antiguas
Plan B con respaldo para cada motor	Se replantea el alcance	Escondió un permiso faltante sin fallar ninguna prueba
Un nodo por motor con memoria mínima	Se replantea	Caída por memoria con 1 M de vectores; kNN vacío por borrados

La regla que más valor dio y que se mantendría por encima de todo: una sola autoridad por dato, y todo lo demás, proyecciones que se pueden reconstruir.

6. Verificación y presentación

- Suite de integración del backend contra los motores reales (sin mocks): 396 pruebas pasan; quedan 3 fallas anteriores a esta entrega (checkout de venta flash y dos de bodegas, introducidas por un cambio de inventario) que se documentan como deuda.
- Invariantes de consistencia I1–I7: se cumplen al cierre de cada fase.
- 56 pruebas de vectores, 50 de permisos y los scripts de evidencia (evaluar_vectores, benchmark_vectores, verificar_permisos) que se pueden volver a ejecutar.

- Entorno DEMO de la presentación actualizado: control de acceso en Redis, MongoDB y Elasticsearch, índice v2 con los vectores de sus 106 productos y el servicio de embeddings dentro del modo presentación (start-presentation.ps1 / stop-presentation.ps1). Verificado: los siete motores en «ok», búsqueda híbrida, similares e inicio funcionando sobre los datos DEMO.

7. Conclusiones

- La recomendación semántica se resolvió sin un motor nuevo: el vector vive en el índice de búsqueda que ya existía, se combina con el texto en una sola consulta y mejora la búsqueda (MRR 0.62 → 0.82) además de habilitar tres funciones nuevas.
- La pregunta de si una base vectorial dedicada vale la pena quedó respondida con números: no a esta escala ni con un millón de productos; sí cuando la memoria de los vectores o su tráfico compitan con la búsqueda de texto, y el camino para llegar ahí ya existe.
- El control de acceso cubre los tres motores NoSQL que usa el backend, con mínimo privilegio por servicio, sin contraseñas en el repositorio y con evidencia que se regenera.
- La retrospectiva identifica con honestidad qué se haría distinto —copias de precio en MongoDB, escritura doble del catálogo, sincronización de Neo4j, Cassandra prematura, seguridad tardía— y por qué.

Anexo. Documentos y comandos

Documento	Contenido
docs/05-diagrama-arquitectura.md	Arquitectura final (diagrama y vistas)
docs/45-ADR-004-busqueda-vectorial.md	Decisión del componente vectorial
docs/46-vectores-implementacion.md	Implementación de los vectores
docs/evidencia/evaluacion-vectores.md	Evaluación de calidad
docs/evidencia/escala-vectores.md	Benchmark de escala
docs/47-control-de-acceso.md	Control de acceso por motor
docs/48-ADR-005-control-de-acceso.md	Decisión del control de acceso
docs/evidencia/permisos.md	Evidencia de permisos
docs/49-retrospectiva.md	Retrospectiva

Comando	Para qué
scripts\setup.ps1	Instalación desde cero (incluye el control de acceso)
scripts\configurar-acceso.ps1	Control de acceso en un equipo ya instalado
scripts\configurar-acceso-demo.ps1 y scripts\migrar-indice-demo.ps1	DEMO: control de acceso e índice con vectores
scripts\start-presentation.ps1 / stop-presentation.ps1	Modo presentación
backend\scripts\verificar_permisos.py	Evidencia del control de acceso
backend\scripts\evaluar_vectores.py	Evaluación de calidad